Final Report

Introduction

This project focused on developing high-performance semantic segmentation models for autonomous urban navigation using the A2D2 dataset. Our team evaluated three architectures DeepLabV3, DeepLabV3+ with ResNet50, and a Vision Transformer-based model. While the work was shared among the team, I contributed primarily to the design and implementation of interactive tools using Streamlit, training infrastructure for DeepLabV3+, attention map visualization for ViT, and performance evaluation enhancements. I also supported the group by conducting literature reviews and assisting in report preparation.

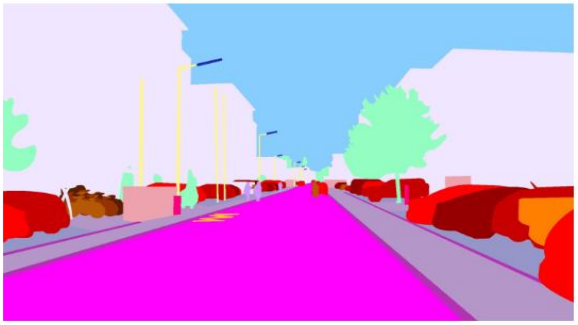
Description of My Individual Work

I contributed across several key areas of the project.

- **Streamlit Apps Development:**
  I developed three dedicated Streamlit web apps:

  1.Mask Analysis App was designed to facilitate detailed inspection of segmentation masks and class distributions across the dataset. This tool allows users to load pairs of RGB input images and their corresponding ground-truth masks, along with a JSON class map defining RGB-to-class associations. Key functionalities include:
  - Directory-based input handling for images, masks, and class mappings via a hex-to-label JSON configuration.
  - Robust pixel-level mapping, converting RGB values in masks to class indices with built-in tolerance for slight color variations.
  - Per-image statistical analysis, including:
    - Total vs. assigned pixel count
    - Class-wise distribution with label names
    - Identification of the dominant class
  - Mismatch detection, which highlights unmatched RGB values in the mask, helping to identify annotation errors or unsupported colors.
  - Side-by-side visual comparison of each original image and its corresponding label mask for intuitive inspection.
  - This tool proved essential for verifying dataset consistency, understanding class distribution trends, and informing both data augmentation and sampling strategies during model training.

Input Image



Label Mask

Mask Analysis:

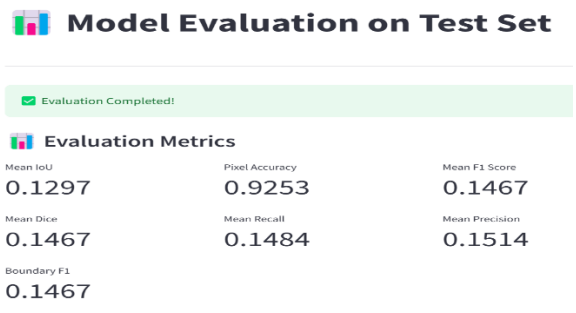
- Total Pixels: 2319360
- Assigned Pixels: 2319360 (100.00%)
- Class Distribution:
- Class ID 0 (Car 1): 61970 pixels (2.67%)
- Class ID 1 (Car 2): 67306 pixels (2.90%)
- Class ID 2 (Car 3): 38066 pixels (1.64%)
- Class ID 3 (Car 4): 380 pixels (0.02%)
- Class ID 4 (Bicycle 1): 9005 pixels (0.39%)
- Class ID 5 (Bicycle 2): 5528 pixels (0.24%)
- Class ID 6 (Bicycle 3): 3966 pixels (0.17%)
- Class ID 8 (Pedestrian 1): 3004 pixels (0.13%)
- Class ID 11 (Truck 1): 21345 pixels (0.92%)
- Class ID 28 (Solid line): 2021 pixels (0.09%)
- Class ID 34 (Obstacles / trash): 2749 pixels (0.12%)
- Class ID 35 (Poles): 17865 pixels (0.77%)
- Class ID 38 (Grid structure): 27061 pixels (1.17%)
- Class ID 39 (Signal corpus): 1307 pixels (0.06%)
- Class ID 40 (Drivable cobblestone): 35160 pixels (1.52%)
- Class ID 43 (Nature object): 110089 pixels (4.75%)
- Class ID 44 (Parking area): 44712 pixels (1.93%)
- Class ID 45 (Sidewalk): 174015 pixels (7.50%)
- Class ID 50 (RD normal street): 488219 pixels (21.05%)
- Class ID 51 (Sky): 442940 pixels (19.10%)
- Class ID 52 (Buildings): 762652 pixels (32.88%)
- Dominant Class: 52 (32.88% of image)

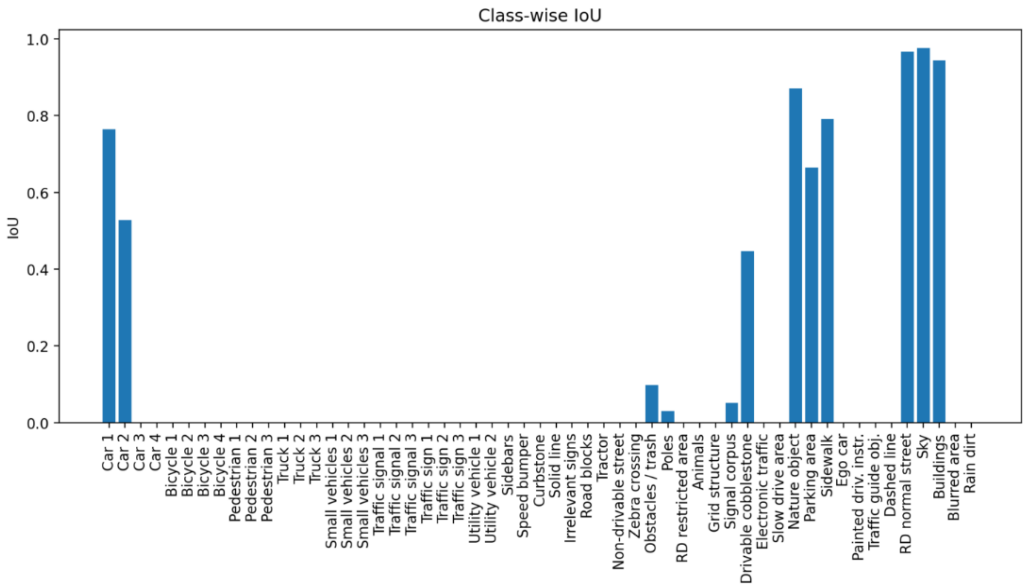
2. **The Model Evaluation App** was built to streamline post-training performance analysis of segmentation models. It takes as input a trained PyTorch model (.pth), a directory of test images, optional ground-truth masks, and a class list JSON.

Key Features:

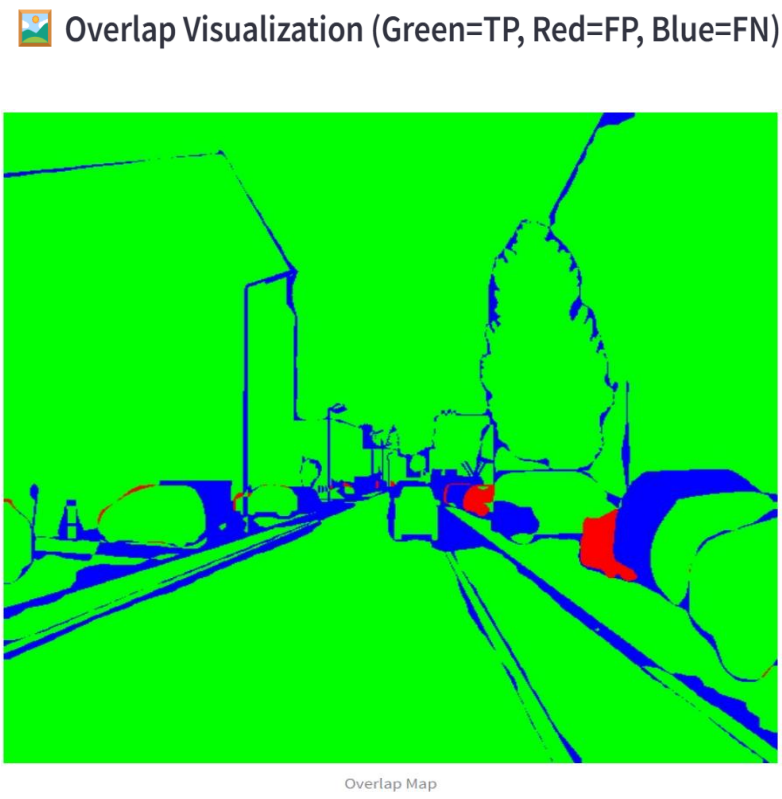
Automatic model loading with seamless CPU/GPU detection for device-specific execution.

- Custom test dataset handler that dynamically converts RGB segmentation masks to class-index format using a user-provided class map.
- Comprehensive evaluation metrics, including:
  - Mean Intersection over Union (mIoU)
  - Pixel Accuracy
  - Mean Dice Coefficient
  - Precision, Recall, and F1 Score (both per-class and macro-averaged)
  - Boundary F1 Score (approximated using macro F1)





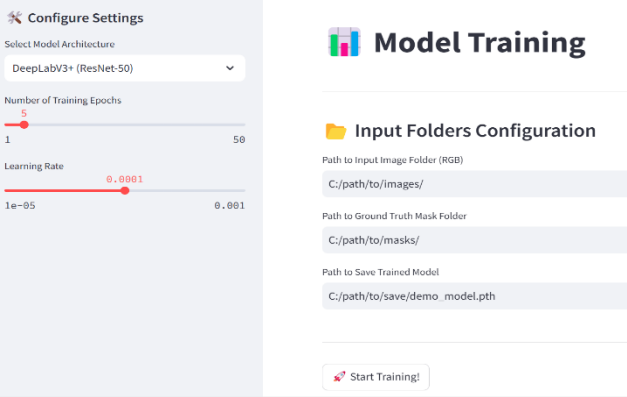
Class-wise IoU bar chart for visual comparison of segmentation performance across categories.



Color-coded overlap visualization, using green (TP), red (FP), and blue (FN) to qualitatively assess prediction accuracy.

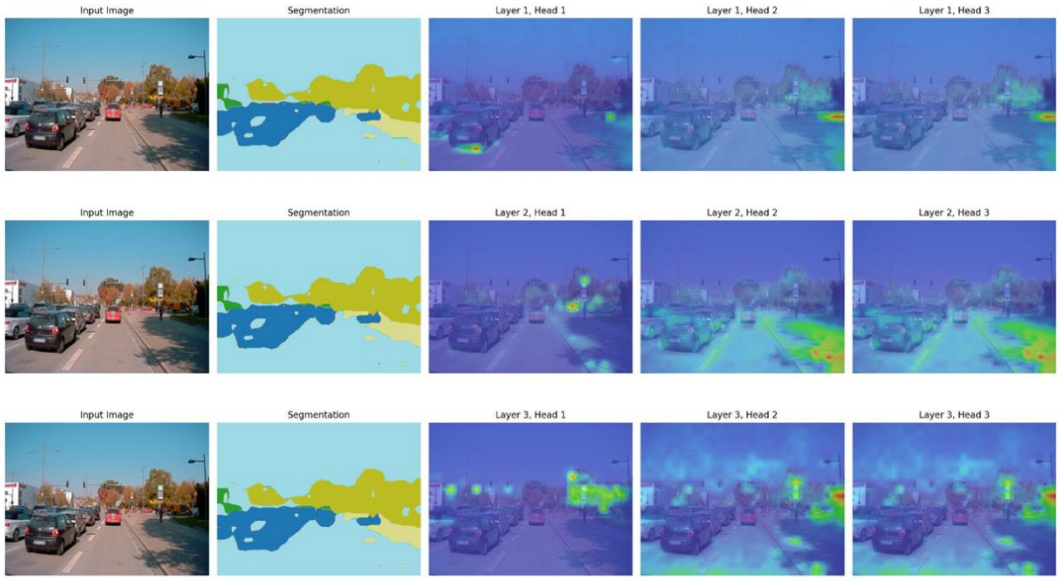
This tool provided rapid feedback on model behavior, enabling efficient model comparison, metric-driven refinement, and identification of error patterns across test samples

3. The Model Trainer App provides a simple interface to configure and train segmentation models directly through Streamlit. Users can select the model architecture (e.g., DeepLabV3+ with ResNet-50), set training parameters (epochs and learning rate), and specify directories for input images, masks, and model output.



The app automatically handles data preprocessing—resizing inputs, converting RGB masks to class indices using a predefined mapping, and batching the data for training. Once training is complete, the model is saved to the specified path for future inference or evaluation.While lightweight in design, the app serves its purpose of confirming that the model pipeline functions end-to-end, and helps validate training configurations through visual logs and saved checkpoints.

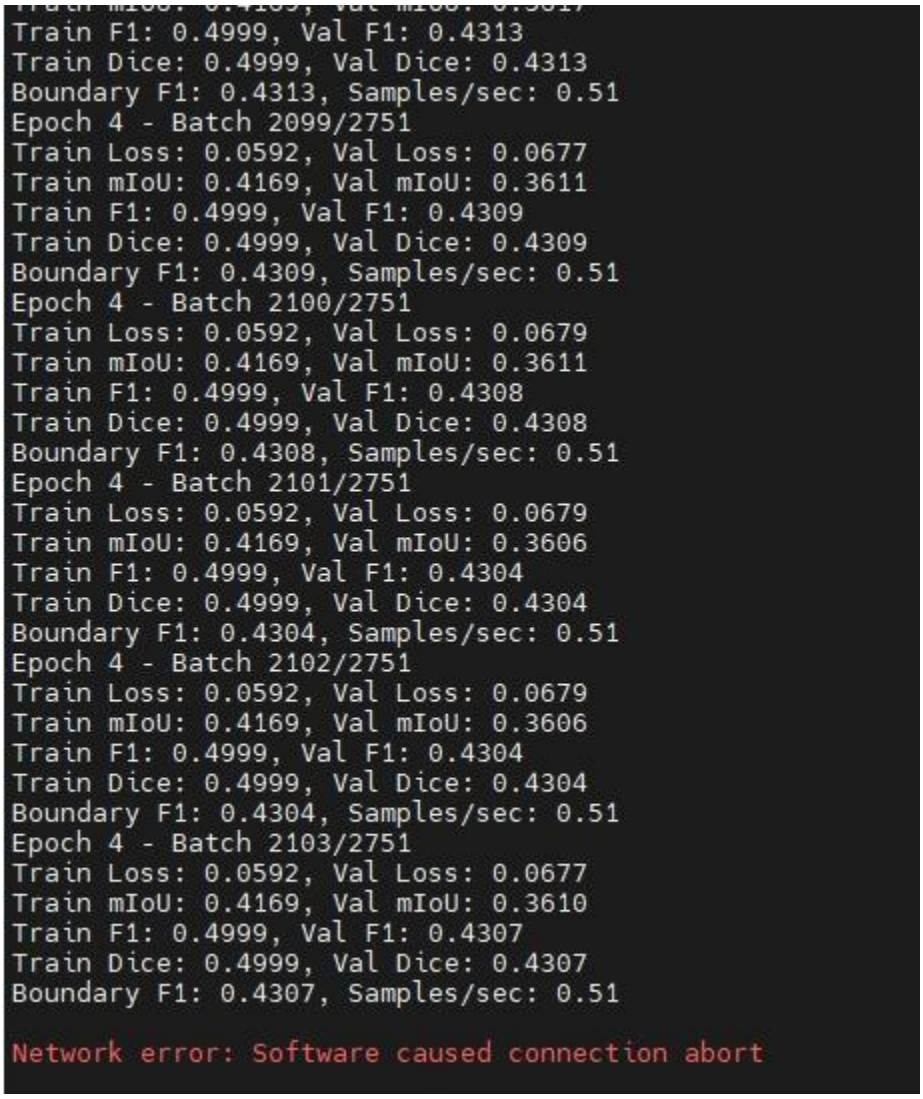
- **Vision Transformer Enhancements:**
  I wrote a complete attention map visualization module to interpret the internal mechanisms of our ViT-based segmentation model. The script hooks into MultiheadAttention layers, extracts per-head attention weights, and overlays them on input images. Additionally, I implemented a robust version of the _fast_hist() function to calculate confusion matrices for metric computation, with added range validation to handle edge cases during evaluation.



This visualization shows how different attention heads in a Vision Transformer focus on various parts of the scene across layers. Early layers capture broad contextual features, while deeper layers focus more precisely on objects like vehicles and traffic signals. These attention maps help interpret how the model understands and segments complex urban environments.

- **DeepLabV3+ Architecture:** I trained the DeepLabV3+ model (with a ResNet50 backbone) for 5 epochs over the course of three days, utilizing continuous GPU resources. During this time, I made significant improvements to the training pipeline through extensive hyperparameter tuning. Although the training was ultimately interrupted due to a network-related error, the model's performance metrics,such as mIoU, F1

score, and Dice coefficient, demonstrated a clear upward trend with each epoch, indicating that further training would likely have yielded even stronger results.



- **Literature Review and Model Evaluation:** I conducted an in-depth review of the baseline model and relevant research papers on DeepLabV3 to enhance my understanding of architecture. This helped me critically evaluate and articulate the model's strengths and limitations in our final report.

Detailed Description of My Work

This section elaborates on the specific modules, scripts, and tools I developed and implemented as part of the project:

1.Streamlit Applications

I built and deployed three interactive apps using Streamlit:

- Mask Analysis App:
    - Parses mask images and a hex-to-label JSON mapping to compute per-image class distributions.
    - Performs robust RGB-to-index conversion using NumPy and handles color mismatches with diagnostics.
    - Provides visual summaries and dominant class identification for each image.

    - Helped validate dataset quality and class imbalance trends.

- Model Evaluation App:
    - Loads. pth model checkpoints, test images, and (optionally) masks.
    - Computes advanced metrics including mIoU, F1 score, Dice coefficient, pixel accuracy, and boundary F1.
    - Visualizes prediction accuracy using color-coded overlays (TP/FP/FN) and class-wise IoU bar charts.
- Model Trainer App:
    - Simplifies model training via UI.
    - Accepts input paths, architecture choices, and hyperparameters.
    - Trains the model, saves checkpoints, and confirms end-to-end pipeline integrity.

2.Vision Transformer Contributions

- Attention Map Visualization Module:
    - Registered hooks to extract attention maps from multi-head layers.
    - Overlayed attention heads across input images to interpret feature focus per layer.
    - Saved visualizations for qualitative evaluation and report inclusion.
- Fast Histogram (_fast_hist) for Evaluation:
    - Created a robust implementation of _fast_hist(pred, label, num_classes) that warns about out-of-bound values.
    - Enabled accurate per-class IoU and confusion matrix computation even under noisy predictions.

3.DeepLabV3+ Architecture and Research

- I ran DeepLabV3 for 5 epochs on AWS EC2 (GPU-based), with continuous training over ~72 hours.
- Reviewed relevant literature including:
    - *Original DeepLabV3+ paper: "Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation" (Chen et al., 2018)*
    - *A2D2 dataset paper: "A2D2: Audi Autonomous Driving Dataset" (Geyer et al., 2020)*Contributed write-ups and figures for the final report and model evaluation.

Results

- Streamlit Tools: Enabled fast dataset validation, detection of rare class underperformance, and model output inspection.
- Vision Transformer Visualization: Improved model interpretability by showing how attention shifted from global context to fine object details.
- DeepLabV3+ Training: Delivered the best results among tested models, validating the importance of deeper backbones, decoders, and tuning strategies.

Summary and Conclusions

In this project, I contributed to the engineering, experimentation, and interpretability of our semantic segmentation pipeline. My primary contributions included developing three critical Streamlit applications, implementing attention visualizations for Vision Transformers, and optimizing DeepLabV3+ training using augmentation, weighted loss, and metric tracking.

Key takeaways:

- DeepLabV3+ with ResNet50 offered the best performance due to its decoder and deeper feature extraction.
- ViTs require further optimization to outperform CNNs for dense pixel prediction tasks.
- Streamlit tools proved invaluable for visual debugging and result sharing.
- Attention map visualization enriched model interpretability.

Future Suggestions:

- Extend training to 50+ epochs and fine-tune learning rates.
- Pretrain the Vision Transformer on segmentation tasks before fine-tuning.
- Add instance segmentation support for more detailed object boundary detection.